Longest Subarray With Sum

AlgoExpert · Hard · Arrays

The problem

- Non-negative ints; [start, end] of the longest subarray summing to target, or [] if none.
- Sliding window.

Intuition

- Values are non-negative, so a sliding window grows to add and shrinks to subtract.

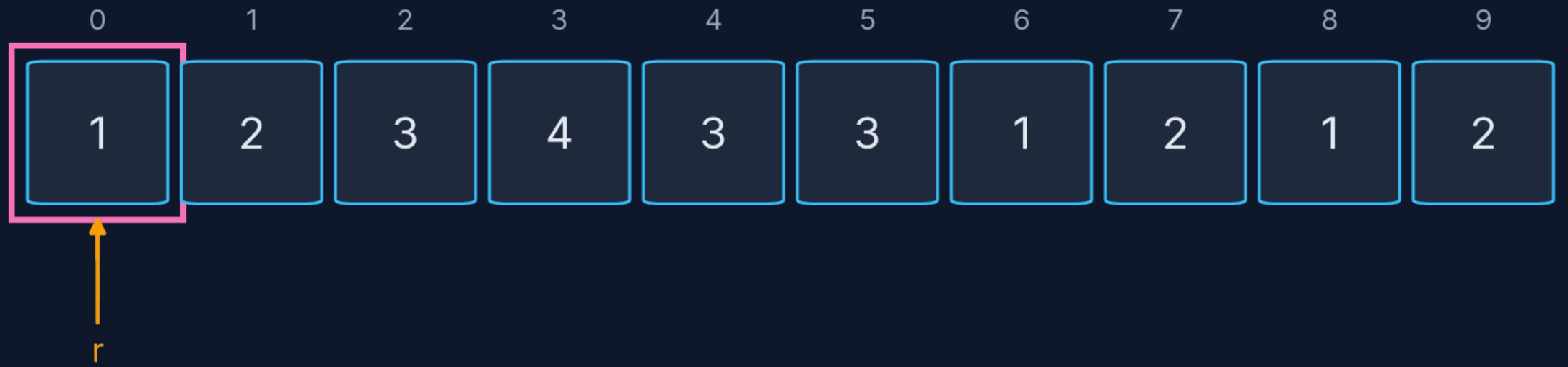
Brute force vs optimal

- Brute force: every subarray → $O(n^2)$
- Sliding window (non-negative values) → $O(n)$
- Prefix sums + hash map for negatives

Approach

- 1. Expand the right edge adding values
- 2. Shrink from the left while the sum is too big
- 3. Record the longest window equal to target

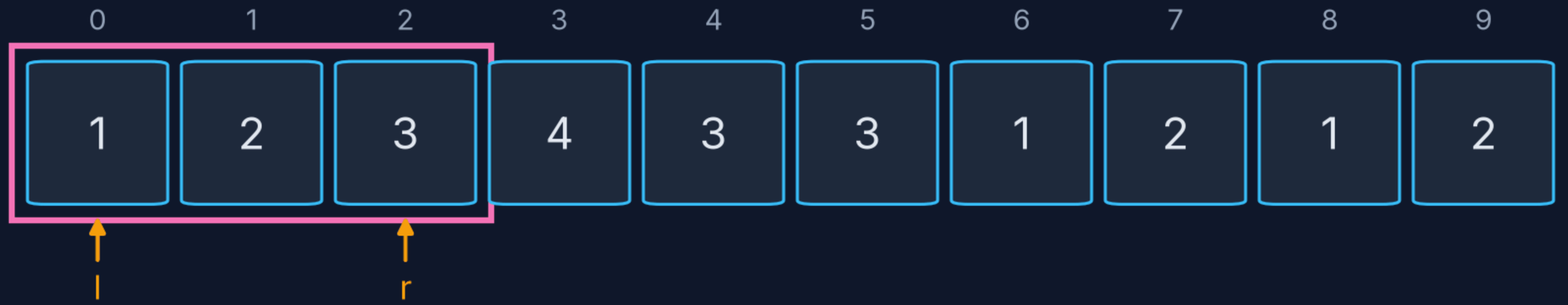
Worked example




```
window sum = 1
best = none
```

r = 0: sum 1

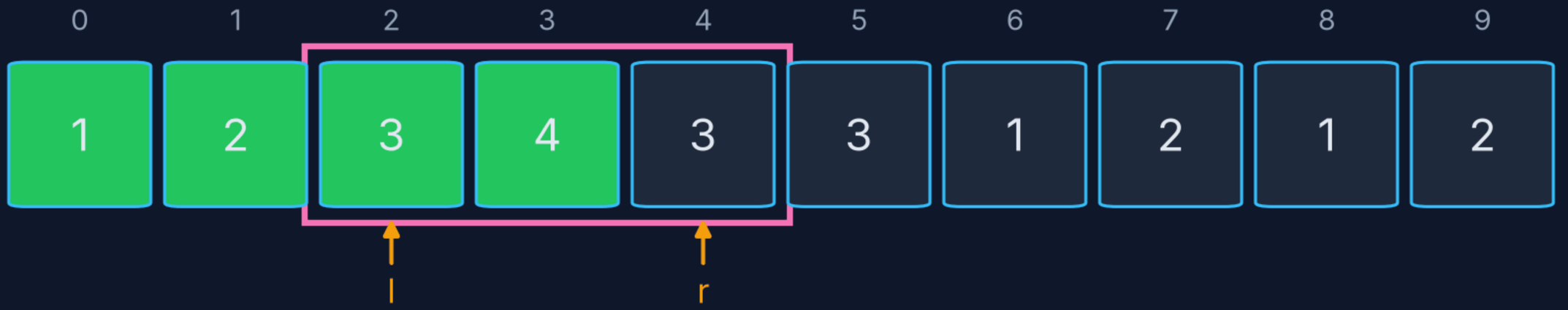
Worked example




```
window sum = 6
best = none
```

r = 2: sum 6

Worked example




```
window sum = 10
best = [0, 3]
```

r = 4: sum 10

Worked example




```
window sum = 10
best = [0, 3]
```

r = 5: sum 10

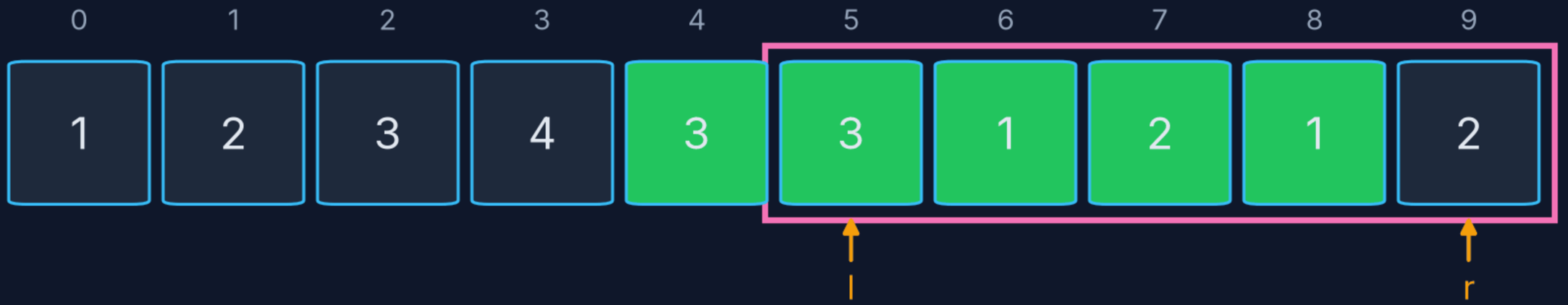
Worked example




```
window sum = 9
best = [0, 3]
```

r = 7: sum 9

Worked example




```
window sum = 9
best = [4, 8]
```

r = 9: sum 9

Java solution

```java
public class LongestSubarrayWithSum {
    public static int[] longestSubarrayWithSum(int[] a, int target) {
        int[] best = {};
        int l = 0, sum = 0;
        for (int r = 0; r < a.length; r++) {
            sum += a[r];
            while (sum > target && l < r) sum -= a[l++];
            if (sum == target && (best.length == 0 || r - l > best[1] - best[0])) best = new int[]{l, r};
        }
        return best;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Window only works for non-negatives
- Zeros extend windows
- No match → []

Interview tips & follow-ups

- Mention the prefix-sum variant

Recap

- Values are non-negative, so a sliding window grows to add and shrinks to subtract.
- Complexity: O(n)